

Методические указания по лабораторной работе (2 часть)

«Виртуальная файловая система /proc»

1. Виртуальная файловая система /proc

Папки (каталоги, англ. directory) и файлы виртуальной файловой системы /proc не хранятся на диске. Они создаются динамически при обращении к ним. Linux предоставляет ядру и модулям ядра дополнительный механизм передачи информации заинтересованным в ней процессам это -файловая система /proc. Первоначально она создавалась с целью получения сведений о процессах (отсюда такое название). Теперь она интенсивно используется и самим ядром, которому есть что сообщить! Например, /proc/modules -- список загруженных модулей, /proc/meminfo -- статистика использования памяти.

Файловая система /proc фактически представляет собой интерфейс ядра, который позволяет получать информацию о процессах и ресурсах, которые они используют. При этом используется стандартный интерфейс файловой системы и системных вызовов. Из этого следует, что управление доступом к адресному пространству осуществляется при помощи обычных прав доступа – на чтение, на запись и выполнение.

Общее взаимодействие с proc заключается в использовании cat и echo из оболочки – командной строки. Например:

```
1 # cat / proc / cpuinfo
2 # echo " 50 " > /proc/sys/kernel / sched_rr_timeslice_ms
```

Данные о каждом процессе хранятся в поддиректории с именем, которым является идентификатор процесса: /proc/<PID>. В поддиректории процесса находятся файлы и поддиректории, содержащие данные о процессе (табл.1):

Таблица – файлы и поддиректории /proc/<PID>

Элемент	Тип	Содержание
cmdline	файл	Указывает на директорию процесса
cwd	символическая ссылка	Указывает на директорию процесса
environ	файл	Список окружения процесса
exe	символическая ссылка	Указывает на образ процесса (на его файл)
fd	директория	Ссылки на файлы, которые «открыл» процесс
root	символическая ссылка	Указывает на корень файловой системы процесса

stat	файл	Информация о процессе
------	------	-----------------------

Процесс может получить свой идентификатор с помощью функции getpid().

Другой способ – использовать ссылку self: /proc/self.

2. Загружаемые модули ядра и виртуальная файловая система /proc

Файлы и поддиректории файловой системы /proc могут создаваться, их можно регистрировать и прекращать их регистрацию. Поэтому /proc часто используются загружаемыми модулями ядра. Файлы и поддиректории файловой системы /proc используют структуру proc_dir_entry: v6.8.7

```
#include <linux/proc fs.h>
```

```

/*
 * This is not completely implemented yet. The idea is to
 * create an in-memory tree (like the actual /proc filesystem
 * tree) of these proc_dir_entries, so that we can dynamically
 * add new files to /proc.
 *
 * parent/subdir are used for the directory structure (every /proc file has a
 * parent, but "subdir" is empty for all non-directory entries).
 * subdir_node is used to build the rb tree "subdir" of the parent.
 */
struct proc\_dir\_entry {
    /*
     * number of callers into module in progress;
     * negative -> it's going away RSN
     */
    atomic\_t in_use;
    refcount\_t refcnt;
    struct list\_head pde\_openers; /* who did ->open, but not ->release */
    /* protects ->pde_openers and all struct pde_opener instances */
    spinlock\_t pde\_unload\_lock;
    struct completion *pde\_unload\_completion;
    const struct inode\_operations *proc\_iops;
    union {
        const struct proc\_ops *proc\_ops;
        const struct file\_operations *proc\_dir\_ops;
    };
    const struct dentry\_operations *proc\_dops;
    union {
        const struct seq\_operations *seq\_ops;
        int (*single\_show)(struct seq\_file *, void *);
    };
    proc\_write\_t write;
    void *data;
    unsigned int state\_size;

```

```

    unsigned int low_ino;
    nlink_t nlink;
    kuid_t uid;
    kgid_t gid;
    loff_t size;
    struct proc_dir_entry *parent;
    struct rb_root subdir;
    struct rb_node subdir_node;
    char *name;
    umode_t mode;
    u8 flags;
    u8 namelen;
    char inline_name[];
} randomize_layout;

```

low_ino : номер inode для директории. Для **proc_register** этот номер должен быть уникальным в файловой системе /proc, значения определены в [<linux/proc fs.h>](#). Для **proc_register_dynamic** номер inode назначаются динамически.

namelen : длина имени

name : уникальное имя виртуального файла (имя данного узла).

mode : тип и права доступа к узлу.

The node's type and permissions. Взяты из [<linux/stat.h>](#).

nlink : число линков к узлу. Инициализировать до 2, если режим включает S_IFDIR, 1 в противном случае.

uid : идентификатор пользователя (uid), которому принадлежит файл (узел), обычно 0.

gid : идентификатор группы (gid), которой принадлежит узел, обычно 0.

size : устанавливает размер узла, значение будет отображаться как размер inode в списках и будет возвращено stat. Если размер не нужен, то его устанавливают равным нулю.

data : непрозрачный указатель, который может использоваться обработчиками proc для передачи локальных данных. Допускается устанавливать свободно при вызове proc_register, обычно NULL. Этот указатель копируется в поле inode.de_generic inode (by proc_get_inode), поэтому он доступен для всех процедур proc, которые передаются inode.

```

struct proc_ops {
    unsigned int proc_flags;
    int      (*proc_open)(struct inode *, struct file *);
    ssize_t  (*proc_read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t  (*proc_read_iter)(struct kiocb *, struct iovec *);
    ssize_t  (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
    /* mandatory unless nonseekable_open() or equivalent is used */
    loff_t   (*proc_lseek)(struct file *, loff_t, int);
    int      (*proc_release)(struct inode *, struct file *);
    poll_t   (*proc_poll)(struct file *, struct poll_table_struct *);
    long      (*proc_ioctl)(struct file *, unsigned int, unsigned long);
#ifdef CONFIG_COMPAT
    long      (*proc_compat_ioctl)(struct file *, unsigned int, unsigned long);
#endif

```

```

int      (*proc_mmap)(struct file *, struct vm_area_struct *);
unsigned long (*proc_get_unmapped_area)(struct file *, unsigned long, unsigned long,
unsigned long, unsigned long);
} _randomize_layout;

```

3. Файловая система /proc: создание файлов, доступных для чтения

Методика работы с файловой системой /proc очень похожа на работу драйверов с файлами устройств: создаётся структура со всей необходимой информацией, включая указатели на функции-обработчики (в нашем случае имеется только один обработчик, который обслуживает чтение файла в /proc). Функция `init_module()` регистрирует структуру, а `module_exit()` отменяет регистрацию (см. лабораторную работу «Загружаемые модули ядра»).

Основная причина, по которой используется `proc_register_dynamic` состоит в том, что номер `inode`, для создаваемого файла, заранее неизвестен, поэтому ядро может определить его самостоятельно, чтобы предотвратить возможные конфликты. В обычных файловых системах, размещенных на диске, не в памяти, как /proc, `inode` указывает на то место в дисковом пространстве, где размещена индексная запись (`index node`, сокращенно -- `inode`) о файле. Inode содержит все необходимые сведения о файле, например права доступа, указатель на первый блок с содержимым файла.

Начиная с ядра 3.10 больше не поддерживается функция `create_proc_entry()`. Вместо нее используются функции:

```

#include <linux/types.h>
#include <linux/fs.h>

extern struct proc_dir_entry *proc_create_data(const char *, umode_t, struct
proc_dir_entry *, const struct file_operations *, void *);

struct proc_dir_entry *proc_create(const char *name, umode_t mode, struct
proc_dir_entry *parent, const struct file_operations *proc_fops);

Дополнительно:
extern struct proc_dir_entry *proc_symlink(const char *,
struct proc_dir_entry *, const char *);

struct proc_dir_entry *_proc_mkdir(const char *, umode_t, struct proc_dir_entry
*, void *, bool);
extern struct proc_dir_entry *proc_mkdir(const char *, struct proc_dir_entry *);

```

Структура `proc_dir_entry` многократно переписывалась: в ранних версиях она содержала поля с функциями `read/write`, затем стали использовать структуру, которая хорошо знакомую из разработки драйверов

- struct file_operations, чтобы определить обратные вызовы (call back) чтения и записи: open(), read(), write(). В настоящее время для этого определена структура proc_ops
Рассмотрим пример, который отображает текущее значение jiffies (счетчик тиков) всякий раз, когда он меняется.

Рассмотрим пример загружаемого модуля с определением функций read/write через struct file_operation, в котором создается файл в файловой системе /proc:

```
1 #include <linux/module.h>
2 #include <linux/moduleparam.h>
3 #include <linux/init.h>
4 #include <linux/kernel.h>
5 #include <linux/proc_fs.h>
6 #include <asm/uaccess.h>
7 #define BUFSIZE 100
8
9
10 MODULE_LICENSE("Dual BSD/GPL");
11 MODULE_AUTHOR( "Liran B.H");
12
13
14 static struct proc_dir_entry * ent;
15
16 static ssize_t mywrite(struct file *file, const char __user *ubuf,size_t count, loff_t *ppos)
17 {
18     printk( KERN_ DEBUG "write handler\n");
19     return - 1;
20 }
21
22 static ssize_t myread(struct file *file, char __user *ubuf,size_t count, loff_t *ppos)
23 {
24     printk( KERN_ DEBUG "read handler\n");
25     return 0;
26 }
27
28 static struct file_operations myops =
29 {
30     .owner = THIS_MODULE,
31     .read = myread,
32     .write = mywrite,
33 };
34
35 static int simple_init(void)
36 {
37     ent= proc_create("mydev", 0660, NULL, & myops);
38     return 0;
39 }
40
41 static void simple_cleanup( void)
```

```

42 {
43     proc_remove(ent);
44 }
45
46 module_init(simple_init);
47 module_exit(simple_cleanup);

```

Если откомпилировать и загрузить этот модуль, то будет создан новый файл **/proc/mydev**.

Можно протестировать функции read и write, используя cat и echo путем просмотра kernel log messages.

```

1 # echo "test" > /proc/mydev
2 bash: echo: write error: Operation not permitted
3
4 # cat /proc/mydev
5 # dmesg | tail -2
6 [ 694.640306] write handler
7 [ 714.661465] read handler

```

Родительский аргумент может быть NULL для корня /proc root или нескольких других значений, в зависимости от того, где нужно разместить файл. В таблице 2 перечислены некоторые другие родительские proc_dir_entrys, которые можно использовать, а также их расположение в файловой системе.

Table 2. Shortcut переменных proc_dir_entry

proc_dir_entry	Filesystem location
proc_root_fs	/proc
proc_net	/proc/net
proc_bus	/proc/bus
proc_root_driver	/proc/driver

Можно также создавать каталоги в файловой системе /proc, используя proc_mkdir(), а также символические ссылки с proc_symlink(). Для простых /proc-записей, для которых требуется только функция чтения, используется

create_proc_read_entry(), которая создает запись /proc и инициализирует функцию read_proc в одном вызове. Прототипы этих функций:

```
extern struct proc_dir_entry *proc_symlink(const char *,  
                                           struct proc_dir_entry *, const char *);
```

```
extern struct proc_dir_entry *proc_mkdir(const char *, struct proc_dir_entry *);
```

```
struct proc_dir_entry *create_proc_read_entry( const char *name, mode_t mode,  
                                              struct proc_dir_entry *base, read_proc_t *read_proc, void *data );
```

Передача данных из режима пользователя в режим ядра и из режима ядра в режим пользователя

Ядро загружено в физическую память (при запуске на выполнение ОС). Процессы имеют защищенные виртуальные адресные пространства.

Отображение виртуальной памяти на физическую осуществляется через таблицы страниц процессов (рис. 1). Аппаратная часть непосредственно выполняет отображение виртуальных страниц на физические, а ядро управляет таблицами и их конфигурацией. Обратите внимание на то, что процесс может иметь большое, но не непрерывное, а «разбросанное» адресное пространство (как показано на рисунке); это означает, что небольшие области (страницы) адресного пространства отображаются на физическую память через таблицы страниц. Это позволяет процессу иметь большое виртуальное адресное пространство, но в физической памяти будут находиться только те страницы, с которыми процессор обращается в текущий момент. При этом страницы загружаются в память «по запросу», когда процессор обращается к странице, которая отсутствует в памяти.

Рисунок 1. Страничные таблицы обеспечивают отображение виртуальных адресов на физические

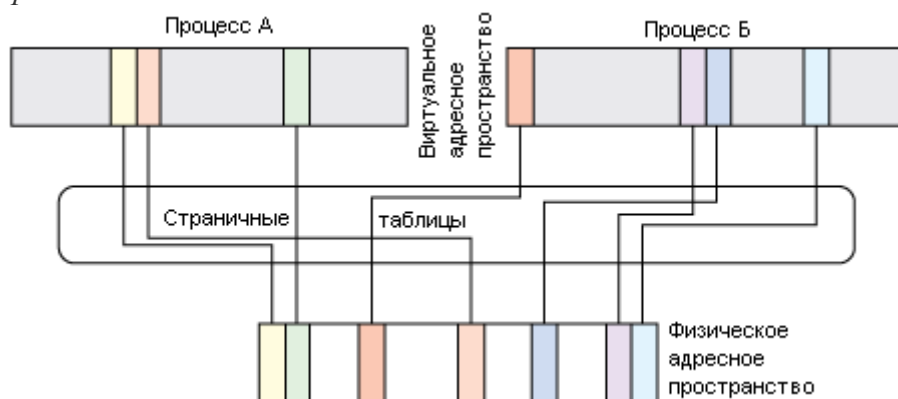


Рис.1

В процессе *замещения страниц* (paging), который в Linux называется *подкачкой* (swap), в соответствии с алгоритмом LRU страницы перемещаются на более медленное устройство хранения (например, на жесткий диск), что позволяет загрузить в память страницы, с которыми необходимо работать в данный момент (рис. 2). Данный подход позволяет размещать в ОЗУ наиболее часто используемые страницы, а наиболее давно не используемые выгружать на жесткий диск. Таким образом, обеспечивается более эффективное использование физической памяти. Заметьте, что некоторые страницы могут ссылаться на файлы – в этом случае грязные (dirty) страницы могут быть вытеснены (replacement) на диск, а чистые (no edit) страницы быть просто помеченными как свободные (копирование на диск не выполняется).

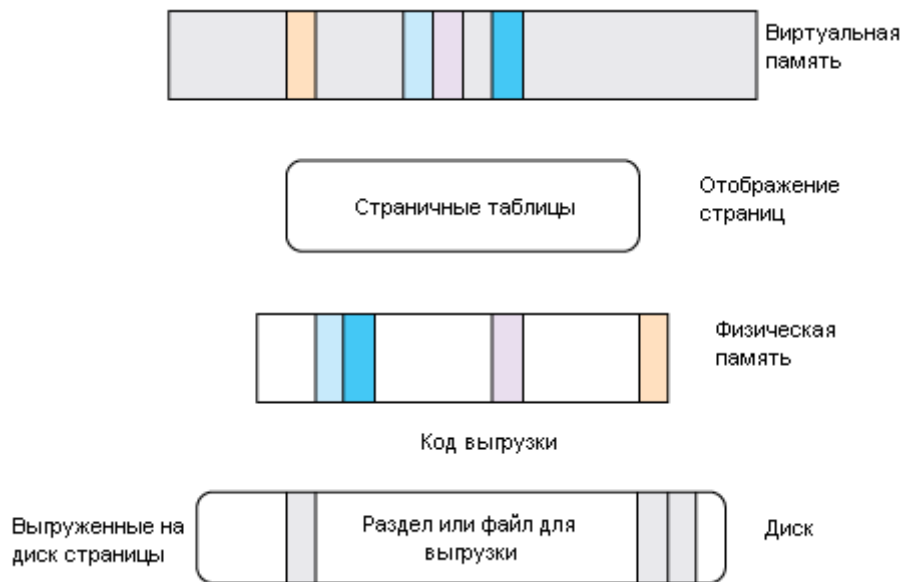


Рис. 2

Процессоры без блока управления памятью (MMU)

Не все процессоры имеют блок управления памятью, поэтому дистрибутив uClinux (версия ОС Linux для микроконтроллеров) поддерживает одноадресное пространство операций. Архитектура без MMU не имеет защиты, обеспечиваемой MMU-блоком, но позволяет ОС Linux работать на процессорах с различными типами архитектур.

Процесс выбора страниц, которые должны быть выгружены на диск, называется *алгоритмом замещения страниц* и может быть реализован по-разному (например, существует алгоритм замещения блока данных с наиболее длительным отсутствием обращений). Этот процесс может начаться в момент обращения к адресу памяти, страница которого не находится в ОЗУ (page fault) (в блоке управления памятью отсутствует ее отображение). [рис. 3](#).

В операционной системе Linux подкачка реализована интересным способом, обладающим рядом полезных свойств. Linux позволяет создавать и использовать несколько разделов подкачки и задавать приоритеты для устройств с различным быстродействием, на которые производится выгрузка данных (например, первоочередным устройством для выгрузки является твердотельный накопитель – SSD-диск, а второстепенным – более медленное устройство). Если SSD-накопителю назначен более высокий приоритет, выгрузка данных будет производиться на него до тех пор, пока на нем не закончится свободное место; лишь после этого страницы памяти будут выгружаться на более медленное устройство с меньшим приоритетом.

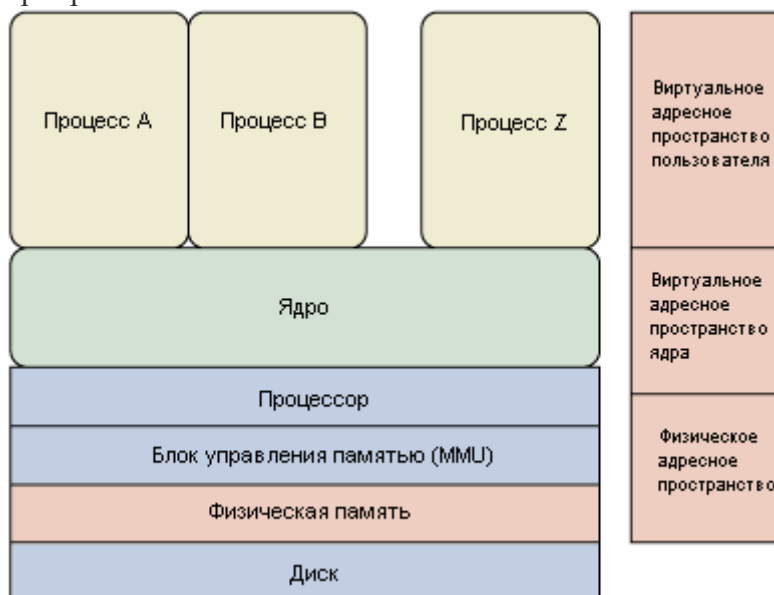


Рис. 3

Не все страницы памяти могут быть выгружены (например, это может быть код ядра, отвечающий за прерывания, или код управления страничными таблицами и логикой подкачки). Очевидно, что такие страницы никогда не должны выгружаться и поэтому являются *закрепленными* (pinned), т. е. постоянно находятся в оперативной памяти. В отличие от страниц ядра, которые никогда не выгружаются, страницы адресного пространства пользователя могут быть выгружены, однако их можно закрепить с помощью функции `mlock` (или `mlockall`). Рассмотрим следующую ситуацию: ядро предположило, что переданный пользователем адрес являлся допустимым и доступным, но по какой-либо причине возникло состояние отсутствия страницы в ядре (например, потому, что пользовательская страница была выгружена). В этом случае может наступить состояние паники ядра. API-функция `mlock` предназначена именно для того, чтобы обеспечивать корректную обработку таких ситуаций.

Все это объясняет необходимость использования специальных функций ядра для передачи данных из ядра в пространство пользователя и из пространства пользователя в пространство ядра.

Для взаимодействия приложений с ядром и ядра с приложениями используются следующие функции:

- **Функция (User Space Memory Access)**

```
unsigned long __copy_to_user ( void __user * to, const void *from, unsigned
                               long n);
```

Формальные параметры

`void __user * to` - Адрес назначения находится в пространстве пользователя.

`const void * from` - Адрес источника находится в пространстве ядра.

`unsigned long n` - количество копируемых байт.

Контекст

Только пользовательский контекст. Эта функция может переводить процесс в состояние *sleep*, если включено *pagefault*.

Описание

Копирует данные из ядра в пространство пользователя. Вызывающий абонент должен проверить указанный блок с помощью `access_ok` до вызова этой функции.

Функция возвращает количество байт, которые не могут быть скопированы. В случае успешного выполнения будет возвращен 0.

- **Функция (User Space Memory Access)**

```
unsigned long __copy_from_user (
```

```
void * to, const void __user *from,  
unsigned long n);
```

Формальные параметры

*void * to* - Адрес назначения находится в пространстве ядра.

*const void __user *from* - адрес источника находится в пространстве пользователя.

unsigned long n - количество копируемых байт.

Контекст

Только пользовательский контекст. Эта функция может переводить процесс в состояние sleep, если включено pagefault.

Описание

Копирует данные из пространства пользователя в пространство ядра. Вызывающий абонент должен проверить указанный заданный блок с помощью `access_ok` до вызова этой функции.

Функция возвращает количество байт, которые не могут быть скопированы. В случае успешного выполнения будет возвращен 0.

Если какие-то данные не могут быть скопированы, эта функция. Если некоторые данные не могут быть скопированы, эта функция добавит нулевые байты к скопированным данным до требуемого размера.

Альтернативная версия - `__copy_from_user_inatomic` - может быть вызвана из атомарного контекста и не сможет «спать». В этом случае не скопированные байты * НЕ * будут заполнены нулями. Смотрите `fs / filemap.h` для объяснения того, почему это необходимо.

```
#ifdef INLINE COPY FROM USER  
static inline __must check unsigned long __copy_from_user(void *to, const void  
__user *from, unsigned long n)  
{  
    unsigned long res = n;  
    might_fault();  
    if (!should fail usercopy() && likely(access_ok(from, n))) {  
        instrument copy from user before(to, from, n);  
        res = raw copy from user(to, from, n);  
        instrument copy from user after(to, from, n, res);  
    }  
    if (unlikely(res))  
        memset(to + (n - res), 0, res);  
    return res;  
}
```

```

#else
extern __must check unsigned long copy from user(void *, const void __user *,
unsigned long);
#endif

#ifdef INLINE COPY TO USER
static inline __must check unsigned long copy to user(void __user *to, const
void *from, unsigned long n)
{
    might fault();
    if (should fail usercopy())
        return n;
    if (access ok(to, n)) {
        instrument copy to user(to, from, n);
        n = raw copy to user(to, from, n);
    }
    return n;
}
#else
extern __must check unsigned long copy to user(void __user *, const void *,
unsigned long);
#endif

static __always inline unsigned long __must check copy from user(void *to,
const void __user *from, unsigned long n)
{
    if (check copy size(to, n, false))
        n = copy from user(to, from, n);
    return n;
}

```

Приведенные функции copy_to_user() и copy_from_user() вызываются в соответствующих точках входа модуля, которые регистрируются с использованием struct proc_ops:

```

static ssize_t fortune_read(struct file *file, char *buf, size_t count, loff_t *f_pos);
static ssize_t fortune_write(struct file *file, const char *buf, size_t count, loff_t
*f_pos);
static int fortune_open(struct inode*, struct file*);
static int fortune_release(struct inode*, struct file*);
static struct proc_ops fops = {
    .proc_read = fortune_read,
    .proc_write = fortune_write,
    .proc_open = fortune_open,
    .proc_release = fortune_release,
};
char *cookie_buf;
struct proc_dir_entry *proc_file;
unsigned int read_index;
unsigned int write_index;
...
int fortune_init(void)
{

```

```
cookie_buf = vmalloc(COOKIE_BUF_SIZE);
if (!cookie_buf)
{
    printk(KERN_INFO "fortune: [ERROR] [can't malloc cookie buffer]\n");
    return -ENOMEM;
}
```

```

memset(cookie_buf, 0, COOKIE_BUF_SIZE);
proc_file = proc_create("fortune", 0666, NULL, &fops);
if (!proc_file)
{
    vfree(cookie_buf);
    printk(KERN_INFO "fortune: [ERROR] [can't create fortune file]\n");
    return -ENOMEM;
}
read_index = 0;
write_index = 0;
proc_mkdir("fortune_dir", NULL);
proc_symlink("fortune_symlink", NULL, "/proc/fortune");
printk(KERN_INFO "fortune: [INFO] [fortune module loaded]\n");
return 0;
}
...

```

Функция sprintf()

int sprintf(char *str, const char *format, ...);

После успешного возврата эта функция возвращает количество напечатанных символов (исключая нулевой байт, используемый для завершения вывода в строки).

Если обнаружена ошибка вывода, возвращается отрицательное значение.

Строка формата - строка символов. Строка формата состоит из нуля или более директив: обычные символы (не %), которые копируются без изменений в выходной поток; и спецификации преобразования, каждая из которых приводит к получению нуля или более последующих аргументов.

Каждая спецификация преобразования вводится символом % и заканчивается спецификатором преобразования. Между ними может быть (в этом порядке) ноль или более флагов, необязательная минимальная ширина поля, необязательная точность и необязательный модификатор длины.

Так как **sprintf** получает строку произвольной длины, ее вызов может привести к переполнению буфера, что невозможно предотвратить. Обратите внимание, что длина строки определена как локально зависящая и ее трудно спрогнозировать. Используйте вместо них **snprintf**.

Перед использованием надо проверить возможность использования функций в конкретной версии ядра.

Данная функция может не поддерживаться разными версиями и подверсиями ядра Linux.

Задание на лабораторную работу

Написать программу – загружаемый модуль ядра (LKM) – которая поддерживает чтение из пространства пользователя и запись в пространство пользователя из пространства ядра.

После загрузки модуля пользователь должен иметь возможность загружать в него строки с помощью команды `echo`, а затем считывать их с помощью команды `cat`.

Функция `init` (например, `init_fortune_module`) выделяет пространство для «горшка с печеньем» (`cookie_pot`) используя `vmalloc()`, а затем очищает его с помощью `memset()`.

Обратите внимание, что на сайтах могут находиться устаревшие версии кода.

```
// устаревшая версия – часто приводится на сайтах
cookie_pot = (char *)vmalloc( MAX_COOKIE_LENGTH );
if (!cookie_pot)
{
    ret = -ENOMEM;
} else
{
    memset( cookie_pot, 0, MAX_COOKIE_LENGTH );
    proc_entry = create_proc_entry( "fortune", 0644, NULL );//начиная с версии 3.10
    //используется proc\_create\_data\(\)
    if (proc_entry == NULL)
    {
        ret = -ENOMEM;
        vfree(cookie_pot);
        printk(KERN_INFO "fortune: Couldn't create proc entry\n");
    } else
    {
        cookie_index = 0;
        next_fortune = 0;
        ...
    }
}
```

Когда `cookie_pot` (буфер, англ. Buffer) выделен и пуст, создается `proc_dir_entry` в корне `/proc root`, который называется, например, `fortune` или `myroot` и т.п.

Файл `cookie_pot` представляет собой страницу длиной (4 КБ) и управляется двумя индексами. Первый, `cookie_index`, определяет, где будет записываться следующий файл `cookie`. Переменная `next_fortune` определяет, где следующий файл `cookie` будет считаться для вывода. Я просто переношу `next_fortune` в начало, когда все состояния были прочитаны.

Запись нового файла `cookie_pot`: если буфер для записи не доступен, то возвращается `-ENOSPC`, который передается пользовательскому процессу. В противном случае это пространство существует, и используется функция `copy_from_user()` для копирования пользовательского буфера непосредственно в файл `cookie_pot`. Затем увеличивается файл `cookie_index` (в зависимости от длины пользовательского буфера) и `NULL` завершает строку. Наконец, возвращается количество символов, фактически записанных в файл `cookie_pot`.

```

ssize_t fortune_write( struct file *filp, const char __user *buff,

                        unsigned long len, void *data ) {
    ...
if (copy_from_user( &cookie_pot[cookie_index], buff, len )) {

    return -EFAULT;
}
    ...
}

```

Чтение: поскольку объявляемый буфер уже находится в пространстве ядра, можно манипулировать им напрямую и использовать `sprintf` (может не поддерживаться ядром и не рекомендуется использовать) или `copy_to_user()` (рекомендуется) для записи. Если индекс `next_fortune` больше, чем `cookie_index` (следующая позиция для записи), то `next_fortune` уменьшается до нуля, что за циклирует буфер. После того, как строка будет записана в пользовательский буфер, индекс `next_fortune` увеличивается на длину последней написанной строки.

Листинг 1. Демонстрация работы «fortune cookie» LKM

```
[root@plato]# insmod fortune.ko
```

```
[root@plato]# echo "Success is an individual proposition. Thomas Watson" > /proc/fortune
```

```
[root@plato]# echo "If a man does his best, what else is there? Gen. Patton" > /proc/fortune
```

```
[root@plato]# echo "Cats: All your base are belong to us. Zero Wing" > /proc/fortune
```

```
[root@plato]# cat /proc/fortune
```

```
Success is an individual proposition. Thomas Watson
```

```
[root@plato]# cat /proc/fortune
```

```
If a man does his best, what else is there? Gen. Patton
```

```
[root@plato]#
```

В программе необходимо создать файл, поддиректорию и символическую ссылку.

При демонстрации работы программы необходимо:

- Продемонстрировать передачу данных из режима пользователя в режим ядра и из режима ядра в режим пользователя с помощью скриншота.
- Показать в каталоге созданный файл, директорию и поддиректорию.

